

Rezolvări Complete și Detaliate

Subiecte Model 2026 - Informatică și Tehnologia Informației
(Definitivat & Concursul de Titularizare)

Cuprins

Examenul Național pentru Definitivare în Învățământ – Model 2026	2
SUBIECTUL I (60 de puncte)	2
1. Structura de date: Listă simplu înlănțuită	2
a) Noțiuni preliminare: Parcurgerea nodurilor	2
b) Descrierea și exemplificarea etapelor pentru cele 4 operații de inserare	2
c) Exemplu de utilizare într-o problemă completă (Inserare Ordonată)	3
2. Sisteme de operare, rețele și securitate	5
a) Noțiuni preliminare:	5
b) Trei tipuri de amenințări informatice:	5
c) Două acțiuni ale unui antivirus la detectarea unui virus:	5
d) Exemplu de program antivirus cunoscut:	6
3. Programare: Generare etichete sală spectacol	6
4. Baze de date: Asociație Festivaluri	7
a) Modelul conceptual (Entități, Atribute, Relații, Forme Normale, Restricții)	7
b) Modelul Fizic (Structura tabelelor)	8
c) Adăugarea datelor de bază (Comenzi SQL)	8
SUBIECTUL al II-lea (30 de puncte)	8
1. Proiectarea unei activități didactice (Alegere Secvență)	8
OPTIUNEA A: Secvența A (Metoda Divide et Impera)	8
OPTIUNEA B: Secvența B (Dispozitive de stocare a datelor)	8
2. Evaluarea rezultatelor școlare: Proba Scrisă	8
Itemi de evaluare:	9
Concursul Național de Ocupare a Posturilor Didactice – Model 2026	10
SUBIECTUL I (30 de puncte)	10
1. Structura de date: Coadă	10
a) Accesul la elemente	10
b) Descriere și Exemplificare pentru o coadă cu 4 elemente	10
1. Reprezentarea Secvențială (Vector fix și indicii front/rear)	10
2. Reprezentarea Înlanțuită (Noduri dinamice)	10
2. Arhitectura sistemelor de calcul: Unitatea Centrală de Prelucrare (UCP)	10
SUBIECTUL al II-lea (30 de puncte)	11
1. Programare Backtracking: Combinări bancnote	11
2. Algoritmi eficienți: Suprapunere Programări Cabinet Medic	12
SUBIECTUL al III-lea (30 de puncte)	13
1. Proiectare Test de Evaluare și Barem (Secvența A)	13
2. Proiectarea unei strategii didactice pentru Secvența B (Animații și Modele 3D) . .	13

Examenul Național pentru Definitivare în Învățământ – Model 2026

SUBIECTUL I (60 de puncte)

1. Structura de date: Listă simplu înlănțuită

a) Noțiuni preliminare: Parcurgerea nodurilor

O listă simplu înlănțuită este o structură de date dinamică formată din noduri alocate necontiguu în memoria Heap. Fiecare nod este format din:

- **Informația utilă** (datele stocate).
- **Legătura/Pointerul** către nodul următor (urm / next).

Structura de definire a unui nod:

C++:

```
struct Nod {
    int info;
    Nod* urm;
};
```

Pascal:

```
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        urm: PNod;
    end;
```

Parcurgerea nodurilor: Pornind de la pointerul de start al listei (prim), se vizitează nodul curent și se înaintează la următorul prin reatribuirea pointerului de parcurgere cu legătura sa: $p = p \rightarrow \text{urm}$ (C++) sau $p := p^{\text{urm}}$ (Pascal), până când pointerul devine NULL / nil.

b) Descrierea și exemplificarea etapelor pentru cele 4 operații de inserare

Presupunem o listă cu 5 noduri: prim $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$ [50] $\rightarrow$ NULL. Noul nod de inserat este indicat de pointerul nou, având valoarea 99.

1. Inserarea înainte de primul nod (la începutul listei)

- **Descriere:** Legătura noului nod este setată să indice către primul nod actual al listei. Pointerul de start al listei (prim) este apoi actualizat pentru a indica spre noul nod.
- **Cod C++:**

```
nou->urm = prim;
prim = nou;
```

- **Cod Pascal:**

```
nou^urm := prim;
prim := nou;
```

2. Inserarea după ultimul nod (la sfârșitul listei)

- **Descriere:** Se parcurge lista până când se ajunge la ultimul nod (cel care are $\text{urm} == \text{NULL}$). Legătura acestui ultim nod se setează să indice către nou, iar legătura lui nou devine NULL.
- **Cod C++:**

```
Nod* p = prim;
while (p->urm != nullptr) p = p->urm;
p->urm = nou;
nou->urm = nullptr;
```

- **Cod Pascal:**

```
p := prim;
while p^.urm <> nil do p := p^.urm;
p^.urm := nou;
nou^.urm := nil;
```

3. Inserarea înainte de un nod intermediar dat (ex. înaintea nodului cu valoarea 30)

- **Descriere:** Este necesară parcurgerea listei pentru a identifica nodul anterior celui căutat (nodul cu valoarea 20). Legătura lui nou este setată să indice către nodul căutat (nou->urm = anterior->urm). Legătura nodului anterior se actualizează să indice către nou.

- **Cod C++:**

```
Nod* ant = prim;
while (ant->urm != nullptr && ant->urm->info != 30) ant = ant->urm;
nou->urm = ant->urm;
ant->urm = nou;
```

- **Cod Pascal:**

```
ant := prim;
while (ant^.urm <> nil) and (ant^.urm^.info <> 30) do ant := ant^.urm;
nou^.urm := ant^.urm;
ant^.urm := nou;
```

4. Inserarea după un nod intermediar dat (ex. după nodul cu valoarea 30)

- **Descriere:** Se identifică nodul dat p (cel cu valoarea 30). Legătura lui nou preia adresa succesorului lui p (nou->urm = p->urm). Legătura lui p este setată să indice către nou.

- **Cod C++:**

```
Nod* p = prim;
while (p != nullptr && p->info != 30) p = p->urm;
nou->urm = p->urm;
p->urm = nou;
```

- **Cod Pascal:**

```
p := prim;
while (p <> nil) and (p^.info <> 30) do p := p^.urm;
nou^.urm := p^.urm;
p^.urm := nou;
```

c) Exemplu de utilizare într-o problemă completă (Inserare Ordonată)

- **Enunț:** Se citesc numere întregi până la introducerea valorii 0. Să se creeze o listă simplu înlănțuită prin inserare ordonată crescător, apoi să se afișeze elementele listei.

- **Implementare C++:**

```
#include <iostream>
using namespace std;
```

```
struct Nod {
    int info;
    Nod* urm;
};
```

```

void inserareOrdonata(Nod* &prim, int val) {
    Nod* nou = new Nod{val, nullptr};
    if (prim == nullptr || prim->info >= val) {
        nou->urm = prim;
        prim = nou;
    } else {
        Nod* curent = prim;
        while (curent->urm != nullptr && curent->urm->info < val) {
            curent = curent->urm;
        }
        nou->urm = curent->urm;
        curent->urm = nou;
    }
}

void afisare(Nod* prim) {
    for (Nod* p = prim; p != nullptr; p = p->urm) {
        cout << p->info << " ";
    }
    cout << endl;
}

int main() {
    Nod* prim = nullptr;
    int x;
    while (cin >> x && x != 0) {
        inserareOrdonata(prim, x);
    }
    afisare(prim);
    return 0;
}

```

• Implementare Pascal:

```

program ListaOrdonata;
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        urm: PNod;
    end;
var
    prim: PNod;
    x: integer;

procedure inserareOrdonata(var prim: PNod; val: integer);
var
    nou, curent: PNod;
begin
    new(nou);
    nou^.info := val;
    nou^.urm := nil;
    if (prim = nil) or (prim^.info >= val) then
    begin
        nou^.urm := prim;
        prim := nou;
    end
end

```

```

else
begin
    curent := prim;
    while (curent^.urm <> nil) and (curent^.urm^.info < val) do
        curent := curent^.urm;
    nou^.urm := curent^.urm;
    curent^.urm := nou;
end;
end;

procedure afisare(p: PNode);
begin
    while p <> nil do
    begin
        write(p^.info, ' ');
        p := p^.urm;
    end;
    writeln;
end;

begin
    prim := nil;
    read(x);
    while x <> 0 do
    begin
        pointer := nil;
        inserareOrdonata(prim, x);
        read(x);
    end;
    afisare(prim);
end.

```

2. Sisteme de operare, rețele și securitate

a) Noțiuni preliminare:

1. **Sistem de calcul:** Componenta fizică (hard) și cea logică (soft) ce conlucrează pentru a permite introducerea, stocarea, procesarea și livrarea datelor.
2. **Sistem de operare:** Programul de bază ce administrează memoria, procesele, componentele hardware și software, oferind o interfață utilizator-mașină.
3. **Rețea de calculatoare:** Un ansamblu de echipamente interconectate pentru partajarea de date, fișiere și resurse hardware.

b) Trei tipuri de amenințări informatice:

1. **Troian:** Se ascunde în programe utile; nu se multiplică singur; deschide breșe de acces (Backdoors).
2. **Ransomware:** Crijtează fișierele și cere răscumpărare în criptomonede.
3. **Vierme (Worm):** Se propagă autonom prin rețele, exploatând breșe de securitate; consumă lățimea de bandă.

c) Două acțiuni ale unui antivirus la detectarea unui virus:

1. **Carantinarea:** Mutarea fișierului într-un folder criptat și izolat.
2. **Ștergerea / Curățarea:** Eliminarea semnăturilor virale sau ștergerea fișierului gazdă.

d) Exemplu de program antivirus cunoscut:

- Bitdefender Antivirus.

—

3. Programare: Generare etichete sală spectacol

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <algorithm>

using namespace std;

void eticheta(int s, int d, int &n) {
    int temp = d;
    int p = 1;
    while (temp > 0) {
        p *= 10;
        temp /= 10;
    }
    n = s * p + d;
}

int main() {
    int m, n;
    if (!(cin >> m >> n)) return 0;

    int A[101][101];
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            int val1, val2;
            eticheta(j, i, val1);
            eticheta(i, j, val2);
            A[i][j] = min(val1, val2);
        }
    }

    ofstream fout("def2025.txt");
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            fout << A[i][j] << (j == n ? "" : " ");
        }
        fout << "\n";
    }
    fout.close();

    return 0;
}
```

Soluție Pascal:

```
program GenerareEtichete;
uses math;

var
    m, n, i, j, val1, val2: integer;
```

```

A: array[1..100, 1..100] of integer;
fout: text;

procedure eticheta(s, d: integer; var n: integer);
var
    temp, p: integer;
begin
    temp := d;
    p := 1;
    while temp > 0 do
    begin
        p := p * 10;
        temp := temp div 10;
    end;
    n := s * p + d;
end;

begin
    readln(m, n);

    for i := 1 to m do
    begin
        for j := 1 to n do
        begin
            eticheta(j, i, val1);
            eticheta(i, j, val2);
            A[i, j] := min(val1, val2);
        end;
    end;

    assign(fout, 'def2025.txt');
    rewrite(fout);
    for i := 1 to m do
    begin
        for j := 1 to n do
        begin
            write(fout, A[i, j]);
            if j < n then
                write(fout, ' ');
            end;
            writeln(fout);
        end;
    end;
    close(fout);
end.

```

4. Baze de date: Asociație Festivaluri

a) Modelul conceptual (Entități, Atribute, Relații, Forme Normale, Restricții)

1. **FESTIVAL**: id_festival (PK), denumire (U, NOT NULL), tematica, site_oficial (NULL).
2. **TIP_EVENT**: id_tip (PK), denumire_tip (U, NOT NULL), spatiu_necesar, nr_max_participanti, varsta_minima.
3. **EVENIMENT**: id_eveniment (PK), id_festival (FK), id_tip (FK), denumire, descriere, data_oroinceput, durata (CHECK > 0), locatie, categorie_public_tinta.

Relații:

- **FESTIVAL - EVENIMENT**: 1:M.
- **TIP_EVENTIMENT - EVENIMENT**: 1:M.

Forme Normale (3NF):

- **1NF**: Valori atomice în toate coloanele.
- **2NF**: Toate coloanele non-cheie depind integral de întreaga cheie primară (chei simple, nu compuse).
- **3NF**: Fără dependențe tranzitive (atributele non-cheie sunt direct dependente doar de cheia primară a tabelului lor).

b) Modelul Fizic (Structura tabelurilor)

Tabelă	Câmp	Tip de date	Restricții
FESTIVAL	id_festival	INT	PK, AUTO_INCREMENT
FESTIVAL	denumire	VARCHAR(100)	UNIQUE, NOT NULL
TIP_EVENTIMENT	id_tip	INT	PK, AUTO_INCREMENT
TIP_EVENTIMENT	denumire_tip	VARCHAR(50)	UNIQUE, NOT NULL
EVENIMENT	id_eveniment	INT	PK, AUTO_INCREMENT
EVENIMENT	id_festival	INT	FK -> FESTIVAL(id_festival)
EVENIMENT	id_tip	INT	FK -> TIP_EVENTIMENT(id_tip)
EVENIMENT	durata	INT	CHECK (durata > 0)

c) Adăugarea datelor de bază (Comenzi SQL)

```
INSERT INTO FESTIVAL (denumire, tematica, site_oficial)
VALUES ('EduCode 2025', 'Inovatie in predarea informaticii', NULL);
```

SUBIECTUL al II-lea (30 de puncte)

1. Proiectarea unei activități didactice (Alegere Secvență)

OPTIUNEA A: Secvența A (Metoda Divide et Impera)

- **Metoda: Problematizarea.**
- **Caracteristici:** Stimulează gândirea analitică prin crearea unei situații-problemă; încurajează participarea activă.
- **Scenariu:** Profesorul propune determinarea maximului dintr-un vector prin divizarea repetată în subvectori simetrici, ghidând elevii să descopere singuri principiul recursivității.

OPTIUNEA B: Secvența B (Dispozitive de stocare a datelor)

- **Metoda: Mozaicul (Jigsaw).**
- **Caracteristici:** Dezvoltă spiritul de echipă; fiecare elev este responsabil de asimilarea și predarea unei subteme.
- **Scenariu:** Elevii sunt împărțiți în grupe de experți (HDD, SSD, Cloud, Unități de măsură), studiază suportul tehnic, iar apoi predau subtemele în grupul de bază.

2. Evaluarea rezultatelor școlare: Proba Scrisă

- **Avantaje:** Evaluare obiectivă, economie de timp, reducerea factorului stres.

- **Limită:** Nu poate evalua deprinderile practice pe calculator.

Itemi de evaluare:

Secvența A (Alegere Multiplă): Alegerea metodei optime pentru căutarea rapidă a unei valori într-un tablou sortat. Răspuns: Căutarea binară.

Secvența B (Răspuns Scurt): Diferența dintre HDD și SSD. Răspuns: Viteza de transfer și rezistența la șocuri (fără piese mobile).

Concursul Național de Ocupare a Posturilor Didactice – Model 2026

SUBIECTUL I (30 de puncte)

1. Structura de date: Coadă

a) Accesul la elemente

Coadă funcționează pe principiul FIFO (First-In, First-Out). Adăugarea se face la sfârșitul cozii (rear), iar extragerea de la început (front).

b) Descriere și Exemplificare pentru o coadă cu 4 elemente

Fie coada în care se inserează pe rând elementele A, B, C, D, urmată de o ștergere.

1. Reprezentarea Secvențială (Vector fix și indicii front/rear)

• Cod C++:

```
char Q[100];
int front = 0, rear = 0;
Q[rear++] = 'A'; // Inserare
char sters = Q[front++]; // Stergere
```

• Cod Pascal:

```
var
  Q: array[0..99] of char;
  front: integer = 0;
  rear: integer = 0;
begin
  Q[rear] := 'A'; rear := rear + 1; // Inserare
  sters := Q[front]; front := front + 1; // Stergere
end;
```

2. Reprezentarea Înlănțuită (Noduri dinamice)

• Cod C++:

```
struct Nod { char info; Nod* urm; };
Nod *prim = nullptr, *ultim = nullptr;
```

• Cod Pascal:

```
type
  PNod = ^TNod;
  TNod = record info: char; urm: PNod; end;
var
  prim: PNod = nil;
  ultim: PNod = nil;
```

2. Arhitectura sistemelor de calcul: Unitatea Centrală de Prelucrare (UCP)

- **Magistrale:** Date (translație date), Adrese (locație RAM), Control (semnale R/W și tact).
- **Componente:** ALU (aritmetică și logică), UCC (comandă și control).
- **Parametri:** Frecvență tact 5.2 GHz, Număr nuclee 16, Cache L3 32 MB.

SUBIECTUL al II-lea (30 de puncte)**1. Programare Backtracking: Combinări bancnote****Soluție C++:**

```

#include <iostream>
#include <algorithm>
using namespace std;

int n;
int c[10], b[10];

void tipar() {
    for (int i = 0; i < n; ++i) {
        if (b[i] > 0) {
            cout << b[i] << "x" << c[i] << (c[i] == 1 ? "leu " : "lei ");
        }
    }
    cout << "\n";
}

void back(int idx, int sum) {
    if (sum == 0) { tipar(); return; }
    if (idx < 0) return;
    int max_b = sum / c[idx];
    for (int bills = max_b; bills >= 0; --bills) {
        b[idx] = bills;
        back(idx - 1, sum - bills * c[idx]);
    }
    b[idx] = 0;
}

int main() {
    int s;
    cin >> s >> n;
    for (int i = 0; i < n; ++i) { cin >> c[i]; b[i] = 0; }
    sort(c, c + n);
    back(n - 1, s);
    return 0;
}

```

Soluție Pascal:

```

program CombinariBancnote;
var
    s, i, j, temp: integer;
    n: integer;
    c, b: array[0..9] of integer;

procedure tipar;
var k: integer;
begin
    for k := 0 to n - 1 do
        begin
            if b[k] > 0 then
                begin
                    if c[k] = 1 then write(b[k], 'x', c[k], 'leu ')

```

```

        else write(b[k], 'x', c[k], 'lei ');
    end;
end;
writeln;
end;

procedure back(idx, sum: integer);
var bills, max_b: integer;
begin
    if sum = 0 then begin tipar; exit; end;
    if idx < 0 then exit;
    max_b := sum div c[idx];
    for bills := max_b downto 0 do
    begin
        b[idx] := bills;
        back(idx - 1, sum - bills * c[idx]);
    end;
    b[idx] := 0;
end;

begin
    read(s, n);
    for i := 0 to n - 1 do begin read(c[i]); b[i] := 0; end;
    for i := 0 to n - 2 do
        for j := i + 1 to n - 1 do
            if c[i] > c[j] then
            begin
                temp := c[i]; c[i] := c[j]; c[j] := temp;
            end;
        back(n - 1, s);
    end.

```

2. Algoritmi eficienți: Suprapunere Programări Cabinet Medic

Soluție C++:

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream fin("titu2025.txt");
    int current, val, rescheduled = 0;
    if (!(fin >> current)) return 0;
    while (fin >> val) {
        if (val == current) rescheduled++;
        else current = val;
    }
    cout << rescheduled << endl;
    return 0;
}

```

Soluție Pascal:

```

program EvaluareSuprapuneri;
var
    fin: text;
    current, val, rescheduled: longint;

```

```

begin
  assign(fin, 'titu2025.txt'); reset(fin);
  if eof(fin) then begin writeln(0); close(fin); halt; end;
  read(fin, current);
  rescheduled := 0;
  while not eof(fin) do
  begin
    read(fin, val);
    if val = current then rescheduled := rescheduled + 1
    else current := val;
  end;
  close(fin);
  writeln(rescheduled);
end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Proiectare Test de Evaluare și Barem (Secvența A)

Item	Tip & Cerință (C++ / Pascal)
Item 1	Alegere Multiplă. Parametri transmiși prin valoare vs referință. Răspuns: B.
Item 2	Completare spații libere: determinare dacă un număr este par.
Item 3	Răspuns Scurt: Avantajele modularizării codului.
Item 4	Întrebare structurată pentru urmărirea recursivității.
Item 5	Subiectiv: Calculul sumei cifrelor impare ale unui număr natural.

2. Proiectarea unei strategii didactice pentru Secvența B (Animații și Modele 3D)

- **Mijloc:** Tinkercad.
- **Metoda:** Învățarea prin descoperire.
- **Activitate:** Crearea și modelarea unei animații de translație 3D.